

Checklist per la prima sessione sulla macchina reale

Scopo: chiudere in una sola sessione i punti che il simulatore non permette di verificare (esito-risposta-payprint.md, capitolo 3 punti 11-13 e capitolo 5). Ogni prova dice **cosa fare, cosa guardare e cosa decide**.

Su quale macchina. Questo elenco è scritto per una macchina **nostra**. Sulla macchina di PayPrint vale guida-prove-macchina-payprint.md, che ha la stessa numerazione ma regole più strette su che cosa si può lanciare: in caso di contrasto, comanda la guida. Lì c'è anche l'ordine di lettura da seguire prima della sessione e il capitolo su come si legge il log del banco. **Le stesse prove, dove si potevano fare, sono già state eseguite sul simulatore** il 16 settembre: prove-banchi-ramo1-2026-09-16.md mostra le righe che il banco scrive quando tutto funziona. **Inserire sempre meno dell'importo richiesto.** Se il denaro inserito raggiunge l'importo, la macchina chiude l'incasso da sola e i pulsanti premuti dopo finiscono su un incasso già chiuso: la prova sembra riuscita e non ha provato niente.

Prima di iniziare

- [] Indirizzo e porta della macchina (in queste note 192.168.1.231:9100)
- [] Contante di prova: qualche moneta e 2-3 banconote diverse
- [] Password del pannello (serve per la chiusura forzata, prova 7) e conferma che l'erogazione PA sia abilitata o meno
- [] Il PC raggiunge la macchina: `Test-NetConnection 192.168.1.231 -Port 9100`
- [] Il **Tap** in mezzo, per avere il traffico grezzo con i tempi: `dotnet run --project PayPrint.PagAmico.Tap -- --listen 9200 --target 192.168.1.231:9100` e nei programmi usare `127.0.0.1:9200` (tranne la prova 10). Il Tap va lanciato in una console interattiva (Visual Studio, profilo 3, o terminale): nella sua console si scrivono i comandi delle prove 6 e 9
- [] Banco WinForms con **diagnostica libreria e registra su file** attivi
- [] A fine sessione: copiare `%LOCALAPPDATA%\PayPrint.PagAmico\logs` e i log del Tap, poi `python strumenti\analizza_log.py`

Mai su questa macchina: Fill (esegue incassi veri) e il gruppo riavvii del collaudo, salvo accordo. Su una macchina **non nostra** (quella di PayPrint) valgono regole più strette, gruppo per gruppo: guida-prove-macchina-payprint.md, capitolo 3.

Per ogni prova annotare: ora, comandi inviati, frame ricevuti (dal log), comportamento fisico della macchina.

1. Collaudo di base, senza soldi

```
dotnet run --project PayPrint.PagAmico.LiveTest -- 127.0.0.1 9200 base
```

- **Guardare:** passi tutti OK; forma di ST (nomi dei campi, sN).
- **Decide:** se la libreria parla con questo firmware. Se fallisce qui, fermarsi.

2. Terminatore e raffica (punto 11)

```
dotnet run --project PayPrint.PagAmico.LiveTest -- 127.0.0.1 9200 base,display --terminatore c  
r --pausa 0  
dotnet run --project PayPrint.PagAmico.LiveTest -- 127.0.0.1 9200 base --terminatore nessuno -  
-pausa 0
```

- **Guardare** nel Tap: comandi a pochi ms l'uno dall'altro, ciascuno con la sua risposta. Nessun "comando senza risposta".
- **Decide:** con CR a 0 ms tutto OK → default della pausa da 80 ms a **0**. Senza terminatore a 0 ms comandi persi → conferma che il CR è obbligatorio.

3. Incasso normale e chiusura con _{CM} (domanda 1)

Banco: incasso di 5,00 €, inserire 2,00 €, poi **[CM]**.

- **Guardare:** quanti frame _{CM} arrivano (uno o due), `errorCode` di ciascuno, `collectedAmount` e `committedAmount` nel frame finale; eventuali _{CM} in ritardo fra gli orfani.
- **Decide:** se la chiusura sul frame con `errorCode` diverso da OK è giusta anche sulla macchina (D1).

4. _{CM} a importo superato (domanda 2)

Incasso di 3,00 €, inserire una banconota da 5, **[CM]** subito.

- **Guardare:** resto erogato o no; `changeCoins`, `changeBanknotes`, `amountPaid`, `amountUnpaid`.
- **Decide:** se il flusso di Giano deve prevedere il rimborso con PA. Risponde anche alla domanda 10 (`amountPaid` comprende il resto?).

5. Annullo con denaro dentro (domanda 3)

Incasso di 10,00 €, inserire 4,00 €, **[AN]**.

- **Guardare:** campi del frame AN — `collectedAmount`, `changeReturn`, `amountUnpaid`. Ripetere con la macchina senza monete per il rimborso, se possibile.
- **Decide:** come l'adattatore riconosce un rimborso incompleto.

6. _{BUSY} e comandi durante l'incasso (domanda 4)

Banco collegato attraverso il Tap. Durante un incasso aperto scrivere nella **console del Tap** ST, poi DS: partono chiusi da CR sulla stessa connessione del banco (la libreria invece blocca gli invii laterali). Le righe T->M del Tap sono i comandi mandati così.

- **Guardare:** risposta testuale BUSY, ER con E100/errorType 99, oppure nulla. La risposta arriva anche al banco: deve finire fra i frame orfani e l'incasso deve restare aperto (è D3, sulla macchina vera).
- **Decide:** se `PagAmicoFrame.IsBusy` riconosce la forma vera.

7. Chiusura forzata dal pannello (domanda 7)

Incasso aperto con denaro inserito, chiusura dal pannello (RESTO o rettangolo in alto a sinistra) + password.

- **Guardare:** frame AN con `halted = TRUE`, denaro reso o no, gesto effettivo sul pannello.
- **Decide:** come Giano distingue la chiusura forzata da un proprio annullo.

8. Errore dopo l'OK (domanda 5)

Se possibile senza danni: incasso aperto e banconota inserita male o rifiutata.

- **Guardare:** arriva un ER che chiude l'incasso, o la macchina resta in IN?
- **Decide:** se l'ER va aggiunto ai frame che chiudono l'incasso.

9. Caduta di rete e riconnessione (domanda 6, punto 13)

Incasso aperto con denaro inserito. Chiudere il banco (il Tap chiude anche la sua connessione verso la macchina), riaprirlo e ricollegarsi **dallo stesso PC** attraverso il Tap. Inserire altro denaro.

- **Guardare:** i parziali e l'esito arrivano sul nuovo socket? Nel banco arrivano come frame orfani, perché la libreria non sa dell'incasso. Per chiudere sul nuovo socket scrivere AN (o CM) nella **console del Tap**: funziona? Collegandosi da un altro PC durante l'incasso: rifiutato?
- **Decide:** la forma dell'API di ripresa di un incasso aperto.

10. Cavo staccato (domanda 8, D2)

Banco collegato **direttamente** a 192.168.1.231:9100, **senza Tap**: con il Tap in mezzo la libreria parla con 127.0.0.1, che non cade mai, e vede la caduta solo quando il Tap chiude la sessione (keepalive del Tap, stessi valori). Diagnostica libreria accesa: alla connessione deve comparire keepalive TCP: prima sonda dopo 10 s, poi ogni 2 s, caduta dopo 5 sonde.

Incasso aperto, staccare il cavo di rete **dal lato della macchina** (o fra switch e macchina) per 1 minuto, riattaccarlo. Staccando il cavo del PC, Windows può chiudere subito le connessioni al cambio di rete, e la prova non misurerebbe il keepalive.

- **Guardare:** dopo quanto la libreria segnala la caduta (keepalive di default 10 s, poi sonde ogni 2 s, 5 sonde: attesa circa 20 s), stato della macchina. Ripetere con il banco Compose (Kotlin, JDK 17 aggiornato) per il keepalive della libreria Kotlin.
- **Decide:** i valori del keepalive TCP da impostare prima di togliere il timeout di 5 minuti.

11. Immagini e stampa diretta (domanda 9)

```
dotnet run --project PayPrint.PagAmico.LiveTest -- 127.0.0.1 9200 immagini,print2 --terminator  
e cr
```

Solo sulla nostra macchina. Il gruppo immagini contiene SF, che **sostituisce il logo permanente**. Su una macchina altrui (PayPrint) fare la prova dal banco con [SI]/[SR] e la stampa, con il consenso: vedi guida-prove-macchina-payprint.md.

- **Guardare:** SF/SI accettati con il CR dopo il pacchetto; PTPRDT con CR/LF nel contenuto stampa correttamente.
- **Decide:** se il terminatore va escluso dopo i pacchetti binari.

Dopo la sessione

- ☐ Riportare gli esiti in `esito-risposta-payprint.md` (tabella del capitolo 1 e punti 11-13)
- ☐ Aggiornare i default: pausa (prova 2), keepalive (prova 10)
- ☐ Aprire le correzioni necessarie nelle due librerie, con test offline che riproducono i frame visti